

FDM vs PINN Full-Field Comparison Notebook Guide

fdm_vs_pinn_full_field_comparison.ipynb 셀별 코드 설명

안형준

1 문서 목적

이 문서는 `fdm_vs_pinn_full_field_comparison.ipynb`의 각 셀이 무슨 역할을 하는지 설명한다. 노트북의 목적은 finite difference method(FDM)로 만든 기준 해와 DeepXDE PINN 예측 해를 같은 $X - \tau$ 전체 격자에서 비교하는 것이다.

$$\theta_{\text{FDM}}(X, \tau) \quad \text{vs.} \quad \theta_{\text{PINN}}(X, \tau)$$

비교 대상은 sensor 위치의 값만이 아니라 전체 temperature field이다. 따라서 이 노트북은 단순히 학습 결과의 β, S, h, q_0 만 보는 것이 아니라, PINN이 공간과 시간 전체에서 FDM 기준장을 얼마나 잘 따라가는지 확인한다.

2 노트북 전체 흐름

셀	종류	역할
1	Markdown	노트북 목적과 비교 항목 설명
2	Code	import, 환경변수, 출력 폴더 설정
3	Markdown	학습 설정 설명
4	Code	iteration, network, collocation point 수 설정
5	Markdown	PINN 학습 단계 설명
6	Code	DeepXDE PINN 학습 및 inverse parameter 출력
7	Markdown	FDM 기준장 생성 단계 설명
8	Code	explicit FDM으로 full-field reference 생성
9	Markdown	PINN 예측 단계 제목
10	Code	PINN을 FDM과 같은 격자에서 평가
11	Markdown	error metric 단계 제목
12	Code	MAE, RMSE, max error, relative L2 계산
13	Markdown	contour 비교 단계 제목
14	Code	FDM, PINN, absolute error contour plot 저장
15	Markdown	time slice 비교 단계 제목
16	Code	특정 시간의 $X - \theta$ profile 비교
17	Markdown	sensor history 비교 단계 제목
18	Code	sensor 위치별 시간 이력 비교
19	Markdown	보고서용 요약 단계 제목
20	Code	최종 수치 결과 출력

3 셀 1: 제목과 목적 설명

첫 번째 셀은 Markdown 셀이다. 노트북이 하는 일을 다음 네 가지로 정의한다.

- FDM 기준장 contour

- PINN 예측장 contour
- absolute error contour
- 시간 slice와 sensor 위치 time history 비교

여기서 “full-field”라는 말은 sensor point 몇 개만 비교하지 않고, 모든 X 위치와 모든 시간 τ 에서 θ 값을 비교한다는 뜻이다.

4 셀 2: import와 실행 환경 설정

두 번째 셀은 노트북 실행에 필요한 package와 project module을 불러온다.

```
os.environ.setdefault("DDE_BACKEND", "pytorch")
os.environ.setdefault("PINN_FORCE_CPU", "1")
os.environ.setdefault("MPLCONFIGDIR", "/private/tmp/matplotlib-cache")
```

- DDE_BACKEND: DeepXDE backend를 PyTorch로 고정한다.
- PINN_FORCE_CPU: M1 Mac에서 MPS/Metal crash를 피하기 위해 CPU 실행을 기본값으로 둔다.
- MPLCONFIGDIR: Matplotlib cache 위치를 writable temp directory로 지정한다.

그 다음 `main_deepxde.py`를 `m`이라는 이름으로 import한다. 이 노트북은 물리 상수, FDM synthetic data 설정, DeepXDE model builder를 직접 다시 정의하지 않고 `main_deepxde.py`의 값을 재사용한다.

```
import main_deepxde as m
```

출력 그림은 다음 폴더에 저장된다.

```
OUTPUT_DIR = Path("outputs/fdm_vs_pinn")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

마지막 print들은 실행 환경과 true parameter를 확인하기 위한 것이다. 특히 τ_{final} , β_{true} , S_{true} 가 기대한 값인지 확인한다.

5 셀 3: 학습 설정 안내

세 번째 셀은 Markdown 셀이다. 노트북 기본값이 빠른 smoke test용임을 설명한다. 보고서용으로는 다음 값을 올리는 것이 적절하다.

설정값	보고서용 예시
ITERATIONS	100000
NUM_DOMAIN	2560
NUM_BOUNDARY	256
NUM_INITIAL	128
NUM_TEST	1000
FDM_N_TIME	201

6 셀 4: 학습 configuration 생성

네 번째 셀은 실제 학습 설정값을 만든다.

```
ITERATIONS = 1000
DISPLAY_EVERY = 100
SEED = 0

NUM_DOMAIN = 512
NUM_BOUNDARY = 64
NUM_INITIAL = 64
NUM_TEST = 200

FDM_NX = 101
FDM_N_TIME = 101
```

ITERATIONS는 Adam optimizer를 몇 step 돌릴지 정한다. DISPLAY_EVERY는 loss와 parameter를 몇 iteration마다 출력할지 정한다. FDM_NX와 FDM_N_TIME은 FDM 기준장을 저장할 공간/시간 해상도이다.

```
args = Namespace(
    iterations=ITERATIONS,
    lr=1.0e-3,
    depth=3,
    width=32,
    num_domain=NUM_DOMAIN,
    num_boundary=NUM_BOUNDARY,
    num_initial=NUM_INITIAL,
    num_test=NUM_TEST,
    sensor_time_points=41,
    display_every=DISPLAY_EVERY,
    lbfgs=False,
)
```

args는 command line argument처럼 생긴 설정 묶음이다. main_deepxde.py의 build_deepxde_model(args)가 이 객체를 받아서 network, 학습점 개수, sensor data 개수 등을 설정한다.

```
m.dde.config.set_random_seed(SEED)
```

random seed를 고정하면 같은 설정에서 결과 재현성이 좋아진다.

7 셀 5: PINN 학습 단계 안내

다섯 번째 셀은 Markdown 셀이다. 이 셀은 실제 학습 구현이 main_deepxde.py의 build_deepxde_model()을 사용한다는 점을 설명한다.

즉, 노트북은 별도의 PINN model을 새로 만드는 것이 아니라 이미 정리된 DeepXDE model builder를 호출한다.

8 셀 6: DeepXDE PINN 학습

여섯 번째 셀은 PINN을 실제로 학습한다.

```
model, raw_beta, raw_s = m.build_deepxde_model(args)
```

build_deepxde_model(args)는 세 가지를 반환한다.

반환값	의미
model	DeepXDE model 객체
raw_beta	학습되는 raw inverse parameter
raw_s	학습되는 raw inverse parameter

raw_beta와 raw_s를 그대로 물리 파라미터로 쓰지 않는다. 실제 β, S 는 softplus를 통과시켜 양수로 만든다.

$$\beta = \text{softplus}(\text{raw_beta}), \quad S = \text{softplus}(\text{raw_s})$$

다음 코드는 Adam optimizer로 model을 compile한다.

```
model.compile(
    "adam",
    lr=args.lr,
    loss_weights=m.LOSS_WEIGHTS,
    external_trainable_variables=[raw_beta, raw_s],
)
```

external_trainable_variables에 raw_beta, raw_s를 넣어야 network weight뿐 아니라 inverse parameter도 같이 학습된다.

```
losshistory, train_state = model.train(
    iterations=args.iterations,
    display_every=args.display_every,
    callbacks=[param_printer],
)
```

param_printer callback은 학습 중 β 와 S 가 어떻게 변하는지 출력한다. 학습 후에는 다음 변환을 통해 실제 물리값으로 바꾼다.

$$h = \beta \frac{kA}{PL^2}$$

$$q_0 = S \frac{k\Delta T_{\text{ref}}}{I_{\text{ref}}^2 L^2}$$

9 셀 7: FDM 기준장 생성 단계 안내

일곱 번째 셀은 Markdown 셀이다. PINN과 비교할 기준 해를 sensor 위치만이 아니라 전체 $X - \tau$ 격자에서 만든다는 점을 설명한다.

이 단계에서 생성되는 기준장은 다음 배열이다.

$$\theta_{\text{FDM}}[n_\tau, n_X]$$

10 셀 8: explicit FDM full-field solver

여덟 번째 셀은 이 노트북의 핵심 비교 기준을 만든다. 푸는 무차원 PDE는 다음과 같다.

$$\frac{\partial \theta}{\partial \tau} = \frac{\partial^2 \theta}{\partial X^2} - \beta_{\text{true}} \theta + S_{\text{true}} i(\tau)^2 g(X)$$

격자 생성

```
x_values = np.linspace(0.0, 1.0, nx, dtype=np.float64)
tau_values = np.linspace(0.0, m.TAU_FINAL, n_time, dtype=np.float64)
```

x_values 는 $0 \leq X \leq 1$ 의 공간 격자이다. tau_values 는 $0 \leq \tau \leq \tau_{final}$ 의 저장용 시간 격자이다.

FDM 시간 간격

```
dx = x_values[1] - x_values[0]
dtau = 0.4 * dx**2
n_steps = math.ceil(m.TAU_FINAL / dtau)
```

explicit FDM에서는 시간 간격이 너무 크면 발산할 수 있다. 그래서 diffusion stability를 고려해 $\Delta\tau = 0.4\Delta X^2$ 를 사용한다.

저장 배열

```
theta = np.zeros(nx, dtype=np.float64)
theta_xx = np.empty_like(theta)
theta_map = np.zeros((n_time, nx), dtype=np.float64)
```

θ 는 현재 시간 step의 온도장이다. θ_{xx} 는 공간 2차 미분 근사값이다. θ_{map} 은 비교와 plot을 위해 저장되는 전체 $X - \tau$ field이다.

열원과 true parameter

```
g_x = np.exp(-((x_values - m.X0) ** 2) / (2.0 * m.SIGMA_X**2))
source_on = float(m.S_TRUE) * g_x
beta_true = float(m.BETA_TRUE)
```

$g(X)$ 는 Gaussian heat source이다. 전류가 켜진 구간에서는 $S_{true}g(X)$ 가 source term으로 들어가고, 꺼진 구간에서는 source term이 0이 된다.

단열 경계조건

코드의 양 끝 경계는 다음과 같이 처리한다.

```
theta_xx[0] = 2.0 * (theta[1] - theta[0]) / dx**2
theta_xx[-1] = 2.0 * (theta[-2] - theta[-1]) / dx**2
```

이는 $X = 0, X = 1$ 에서

$$\frac{\partial \theta}{\partial X} = 0$$

인 단열 경계조건을 ghost point 방식으로 반영한 형태이다.

시간 전진

```
theta = theta + step_dtau * (theta_xx - beta_true * theta + source)
```

이 식은 explicit Euler time stepping이다. 현재 온도장 θ^n 에서 다음 온도장 θ^{n+1} 을 직접 계산한다.

출력

함수는 다음 세 개를 반환한다.

반환값	의미
x_values	공간 격자
tau_values	시간 격자
theta_map	FDM 기준 온도장

마지막으로 실제 시간 plot을 위해 무차원 시간 τ 를 초 단위 시간으로 다시 바꾼다.

```
time_seconds = tau_values * m.L**2 / m.ALPHA
```

11 셀 9: PINN 예측 단계 안내

아홉 번째 셀은 Markdown 셀이다. FDM과 같은 $X-\tau$ grid에서 PINN 예측값을 계산한다는 단계 제목이다.

12 셀 10: PINN을 같은 격자에서 평가

열 번째 셀은 FDM이 저장된 격자와 동일한 위치에서 PINN을 평가한다.

```
TAU, X = np.meshgrid(tau_values, x_values, indexing="ij")
xtau_eval = np.column_stack([X.reshape(-1), TAU.reshape(-1)]).astype("float32")
```

meshgrid는 X 와 τ 의 2D grid를 만든다. model.predict()는 입력을 $(N, 2)$ 형태로 받기 때문에, 2D grid를 flatten해서 xtau_eval로 바꾼다.

```
theta_pinn = model.predict(xtau_eval).reshape(len(tau_values), len(x_values))
```

예측 후에는 다시 (n_τ, n_X) 배열로 reshape한다. 이렇게 해야 theta_fdm과 같은 shape이 되어 직접 뺄 수 있다.

13 셀 11: error metric 단계 안내

열한 번째 셀은 Markdown 셀이다. 다음 셀에서 full-field error metric을 계산한다는 제목이다.

14 셀 12: full-field error metric 계산

열두 번째 셀은 PINN 예측장과 FDM 기준장의 차이를 계산한다.

```
error = theta_pinn - theta_fdm
abs_error = np.abs(error)
```

계산하는 metric은 네 개다.

Metric	의미
MAE	평균 절대 오차
RMSE	root mean square error
Max error	전체 격자에서 가장 큰 절대 오차
Relative L2	FDM field norm 대비 error field norm

Relative L2는 다음과 같다.

$$\text{Relative L2} = \frac{\|\theta_{\text{PINN}} - \theta_{\text{FDM}}\|_2}{\|\theta_{\text{FDM}}\|_2}$$

15 셀 13: contour 비교 단계 안내

열세 번째 셀은 Markdown 셀이다. FDM, PINN, absolute error를 contour plot으로 비교한다는 제목이다.

16 셀 14: full-field contour plot

열네 번째 셀은 세 개의 contour plot을 한 figure에 그린다.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 4.5), constrained_layout=True)
```

세 subplot의 의미는 다음과 같다.

Plot	의미
FDM reference	FDM으로 계산한 기준 온도장
PINN prediction	PINN이 예측한 온도장
Absolute error	$ \theta_{\text{PINN}} - \theta_{\text{FDM}} $

그림은 다음 파일로 저장된다.

```
outputs/fdm_vs_pinn/fdm_pinn_full_field_contours.png
```

보고서에서는 이 그림이 전체 field 비교의 핵심 figure가 된다.

17 셀 15: time slice 비교 단계 안내

열다섯 번째 셀은 Markdown 셀이다. 특정 시간에서 X 방향 온도 profile을 비교한다는 제목이다.

18 셀 16: selected time slice 비교

열여섯 번째 셀은 네 개 시간에서 FDM과 PINN의 $X - \theta$ profile을 비교한다.

```
slice_times = [300.0, 600.0, 900.0, 1200.0]
```

각 시간은 current profile의 전류 on/off 전환과 관련 있다.

시간	의미
300 s	첫 번째 heating 구간 끝
600 s	첫 번째 cooling 구간 끝
900 s	두 번째 heating 구간 끝
1200 s	전체 해석 시간 끝

```
idx = int(np.argmax(np.abs(time_seconds - target_time)))
```

위 코드는 원하는 실제 시간에 가장 가까운 FDM 저장 index를 찾는다. 그 다음 같은 index에서 FDM과 PINN profile을 함께 plot 한다.

그림은 다음 파일로 저장된다.

```
outputs/fdm_vs_pinn/fdm_pinn_time_slices.png
```

19 셀 17: sensor history 비교 단계 안내

열일곱 번째 셀은 Markdown 셀이다. sensor 위치에서 시간에 따른 온도 이력을 비교한다는 제목이다.

20 셀 18: sensor 위치별 time history 비교

열여덟 번째 셀은 sensor 위치 $X = 0.25, 0.55, 0.85$ 에서 FDM과 PINN의 시간 이력을 비교한다.

```
for ax, sensor_x in zip(axes, m.SENSOR_X):
    idx = int(round(sensor_x * (len(x_values) - 1)))
```

sensor_x는 무차원 위치이므로, len(x_values)-1을 곱해 가장 가까운 grid index로 변환한다.

```
ax.plot(time_seconds, theta_fdm[:, idx], label="FDM", linewidth=2)
ax.plot(time_seconds, theta_pinn[:, idx], "--", label="PINN", linewidth=2)
```

이 plot은 기존 inverse PINN 학습에서 사용한 sparse sensor 위치에서 두 해가 얼마나 잘 맞는지 보여준다. 그림은 다음 파일로 저장된다.

```
outputs/fdm_vs_pinn/fdm_pinn_sensor_histories.png
```

21 셀 19: 보고서용 요약 단계 안내

열아홉 번째 셀은 Markdown 셀이다. 마지막 셀에서 보고서에 옮기기 쉬운 형태로 주요 수치를 출력한다는 제목이다.

22 셀 20: 최종 summary 출력

마지막 셀은 학습 설정, inverse parameter, error metric을 한 번에 출력한다.

```
print(f"iterations_{iterations}:_{iterations}")
print(f"network_{args.depth}_{args.width}")
print(f"loss_{weights}_{m.LOSS_WEIGHTS}")
```

학습 설정을 기록하는 이유는 결과가 iteration 수, network 크기, loss weight에 따라 달라지기 때문이다.

```
print(f"beta_learned_{beta_learned:.6f}/_{true}_{m.BETA_TRUE:.6f}")
print(f"S_learned_{s_learned:.6f}/_{true}_{m.S_TRUE:.6f}")
print(f"h_learned_{h_learned:.6f}/_{true}_{m.H_TRUE:.6f}")
print(f"q0_learned_{q0_learned:.6e}/_{true}_{m.Q0_TRUE:.6e}")
```

이 부분은 inverse problem이 true parameter를 얼마나 잘 복원했는지 보여준다.

```
print(f"MAE_{mae:.6e}")
print(f"RMSE_{rmse:.6e}")
print(f"Max_error_{max_error:.6e}")
print(f"Relative_L2_{rel_l2:.6e}")
```

이 부분은 full-field prediction accuracy를 수치로 정리한다.

23 보고서에 쓸 때 핵심 포인트

이 노트북 결과를 보고서에 넣을 때는 다음 세 가지를 함께 제시하는 것이 좋다.

1. learned inverse parameter와 true parameter 비교
2. FDM/PINN/error contour figure
3. MAE, RMSE, relative L2 같은 full-field error metric

단순히 h, q_0 가 true value와 비슷하다고 끝내면 부족하다. PINN이 PDE solution field 자체도 잘 재현하는지 보여줘야 하므로 full-field comparison이 필요하다.